# Practice Set: Module 01 — Introduction to C++ & Object-Oriented Programming

## Section 1: Conceptual & Theory Questions (1–10)

**Question 1:** Define what a programming paradigm is, and briefly state why writing code without a structured paradigm leads to poor software design.

**Question 2:** Differentiate between the **Imperative Paradigm** and the **Declarative Paradigm**.

**Question 3:** Compare **Procedure-Oriented Programming (POP)** and **Object-Oriented Programming (OOP)** across the following criteria: Design Flow, Data Security, Access Specifiers, and Code Reuse.

**Question 4:** Explain why POP programs are generally more vulnerable to unintended data modifications compared to OOP programs.

**Question 5:** Define a **Class** and an **Object**. Discuss the difference in memory allocation between a class definition and an object instantiation.

**Question 6:** Detail the six core principles of Object-Oriented Programming covered in the document.

**Question 7:** Differentiate between **Compile-Time Polymorphism** and **Run-Time Polymorphism** with examples of how each is implemented in C++.

**Question 8:** Outline the history and origin of C++. Who created it, at what location, and why was it originally named "C with Classes"?

**Question 9:** Explain the **Zero-Overhead Principle** and why C++ is categorized as a "middle-level language."

**Question 10:** Compare a **Compiler** and an **Interpreter** in terms of translation mechanism, execution speed, output file generation, and error detection timing.

## Section 2: Technical & Interview / Viva Questions (11–20)

**Question 11 (Viva):** How many bytes of memory are allocated in RAM when a class definition is loaded into memory versus when an instance of that class is instantiated?

**Question 12 (Viva):** What is the etymological origin of the name "C++"?

**Question 13 (Viva):** What is "Spaghetti Code," and which programming paradigm flaw typically leads to it?

**Question 14 (Viva):** Is C++ fully backward compatible with C? Can you execute C code using a C++ compiler?

**Question 15 (Viva):** Explain how Encapsulation and Data Hiding protect sensitive object states in C++.

**Question 16 (Viva):** What compiler options/flags are recommended during development to catch latent bugs before runtime?

**Question 17 (Viva):** How does the execution flow of a compiled language like C++ differ from an interpreted language like Python?

**Question 18 (Viva):** Name two programming languages based on Procedure-Oriented Programming (POP) and two based on Object-Oriented Programming (OOP).

**Question 19 (Viva):** What is Abstraction in OOP, and how does a const getter method exemplify it?

**Question 20 (Viva):** Why did Bjarne Stroustrup choose to combine features of Simula 67 with C rather than simply using Simula 67 directly?

## Answers & Explanations

**Answer: 01** A programming paradigm defines the fundamental style, philosophy, and approach used to structure, organize, and execute instructions in a computer program. Unstructured code leads to uncontrolled complexity ("spaghetti code"), poor readability/maintainability, high bug frequency due to unintended side effects, and testing hurdles due to tightly coupled components.

**Answer: 02**

**Imperative Paradigm:** Focuses on *how* to do a task by executing sequential statements that directly mutate memory and program state (e.g., POP, OOP).

**Declarative Paradigm:** Focuses on *what* to accomplish without specifying step-by-step control flow (e.g., Functional Programming, Logic/Prolog, SQL).

**Answer: 03**

**Design Flow:** POP uses a *Top-Down approach*; OOP uses a *Bottom-Up approach*.

**Data Security:** Low in POP (global data is exposed); High in OOP (data abstraction/encapsulation hide state).

**Access Specifiers:** POP has none; OOP supports public, private, and protected.

**Code Reuse:** Low in POP; High in OOP via Inheritance and Polymorphism.

**Answer: 04** POP relies heavily on global variables so multiple functions can process shared data. Lacking access control specifiers, any function in the codebase can access and modify global memory, leaving procedural programs vulnerable to unauthorized modifications.

**Answer: 05**

**Class:** A user-defined blueprint containing data members and member functions. Allocates **0 bytes** of RAM at runtime (it acts only as a schema).

**Object:** A runtime instance of a class. Instantiation allocates physical RAM space to store its attributes (e.g., an int + double allocates ~12 bytes in RAM).

**Answer: 06**

**Class:** User-defined blueprint/schema.

**Object:** Runtime instance of a class.

**Encapsulation:** Wrapping data members and functions into a single class container while restricting access.

**Abstraction:** Exposing essential interfaces while hiding background execution mechanisms.

**Inheritance:** Enabling a derived class to inherit attributes/behaviors from a base class.

**Polymorphism:** Allowing a single interface, function, or operator to exhibit different behaviors.

**Answer: 07**

**Compile-Time Polymorphism:** Resolved during compilation; realized via Function Overloading and Operator Overloading.

**Run-Time Polymorphism:** Resolved dynamically during execution; realized via Function Overriding using Virtual Functions.

**Answer: 08** Created by Bjarne Stroustrup at AT&T Bell Laboratories in Murray Hill, New Jersey, between 1979–1980. It was originally named "C with Classes" because it combined Simula 67's object-oriented features with C's speed and direct memory access.

**Answer: 09**

**Zero-Overhead Principle:** C++ features are designed so that you do not pay extra memory or execution time penalties for features you do not explicitly use.

**Middle-Level Language:** It combines high-level abstractions (classes, STL, exception handling) with low-level capability (direct memory manipulation via pointers and explicit allocation).

**Answer: 10**

**Translation Mechanism:** Compilers translate the whole source file at once; Interpreters translate statement-by-statement at runtime.

**Execution Speed:** Compiled binaries run blazingly fast; Interpreted execution is slower due to runtime translation overhead.

**Output File:** Compilers generate standalone native binaries (.exe/ELF); Interpreters create no output executable.

**Error Detection:** Compilers catch errors at Compile-Time; Interpreters halt at Runtime when reaching the error line.

**Answer: 11** A class definition allocates **0 bytes** of RAM because it is merely a type definition/schema. Memory is allocated only when an object is instantiated.

**Answer: 12** The name comes from C's unary increment operator (++), signifying that C++ is an "incremented" or enhanced evolution of C.

**Answer: 13** "Spaghetti Code" refers to unstructured, unpredictable execution paths caused by a lack of modular architecture (typical of unstructured procedural programming).

**Answer: 14** Yes, C++ is a superset of C, so almost all valid C code will compile under a C++ compiler. However, modern idiomatic C++ favors classes, references, dynamic STL containers, and smart pointers.

**Answer: 15** Encapsulation binds data and member functions inside a class container, marking sensitive variables as private. Access is provided strictly through validation methods (setters/getters), preventing external code from corrupting state.

**Answer: 16** Recommended compiler diagnostics include: g++ -Wall -Wextra -std=c++20 main.cpp.

**Answer: 17** A compiled language scans the source file completely, generating a native machine code binary saved on disk for direct CPU execution. An interpreted language translates source code line-by-line in real time during runtime.

**Answer: 18**

**POP:** C, FORTRAN, Pascal, BASIC.

**OOP:** C++, Java, C#, Python, Simula.

**Answer: 19** Abstraction hides internal complexity and exposes only essential interfaces. A const getter method hides how the internal variable is stored or calculated, returning only the requested value safely.

**Answer: 20** While Simula 67 offered great OOP software structures, it was too slow for high-performance applications. C provided hardware speed and direct memory access, so Stroustrup combined both to get the best of both worlds